IST604 – Web Services & Distributed Computing

Chapter 3: Developing Web Services Using SOAP — Full Model Answers

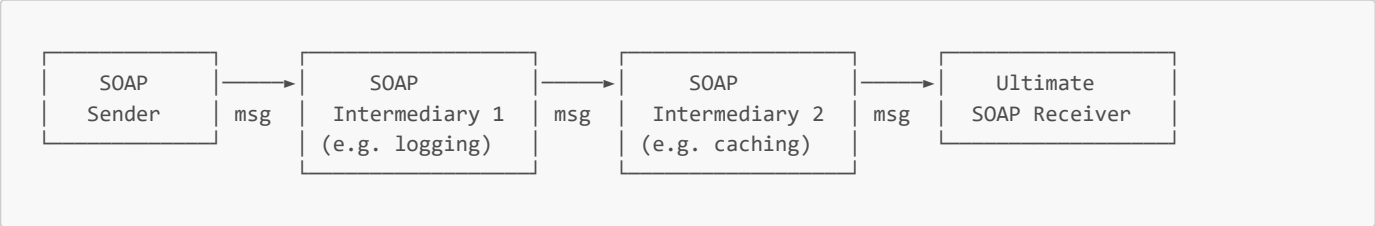
Part I: Structural / Short-Answer Questions

Q1. Describe the three roles involved in SOAP message transmission and explain the function of each. Include a diagram showing the message path. *(6 marks)*

Answer:

SOAP message transmission is not always a direct point-to-point exchange. The message delivery path can involve intermediary nodes between the original sender and the final destination. Three distinct roles are defined:

Diagram:



Role 1 — SOAP Sender:

The SOAP Sender is the node that creates and initiates the SOAP message. It constructs a well-formed SOAP Envelope containing the appropriate Header (optional metadata) and Body (payload/operation data), and transmits it toward the intended destination using a transport protocol such as HTTP. The Sender is responsible for the content and structure of the message.

Role 2 — SOAP Intermediary:

A SOAP Intermediary is an optional processing node positioned along the message delivery path between the Sender and the Ultimate Receiver. There can be zero, one, or multiple intermediaries in a chain. Each intermediary receives the SOAP message, performs some processing on it (such as filtering, security enforcement, logging, caching, routing, or protocol bridging), and then forwards the (possibly modified) message to the next node in the chain. Intermediaries may process specific Header blocks directed at them and are not the final intended consumer of the message.

Role 3 — Ultimate SOAP Receiver:

The Ultimate SOAP Receiver is the final intended destination of the SOAP message — the node the Sender originally addressed the message to. Unlike intermediaries, the Ultimate Receiver fully processes the Body of the SOAP message, executes the requested operation (business logic), and generates the corresponding SOAP response message to send back to the original Sender.

Q2. Explain the role of WSDL in a SOAP-based web services architecture. What three questions does a WSDL document answer about a web service? *(4 marks)*

Answer:

Role of WSDL:

WSDL (Web Services Description Language) serves as the **formal contract and metadata language** in a SOAP-based web services architecture. It is an XML-based document that formally describes a web service in a machine-readable format. WSDL sits at the centre of the web services model: it is what the Service Provider publishes to the UDDI registry, what the Service Requester retrieves during discovery, and what tools like `wsimport` read to automatically generate client stubs. Without WSDL, a client developer would need to manually read documentation and hand-craft SOAP messages — WSDL automates and standardises this process.

WSDL describes how service providers and requesters communicate with one another by defining the binding mechanisms used to attach protocols, data formats, and abstract messages to the set of endpoints that define the location of the service.

Three Questions WSDL Answers:

1. **WHAT** — What operations (methods) does the service expose, and what data types do they use? (Defined in `<portType>`, `<message>`, and `<types>` elements.)
2. **WHERE** — Where is the service located? What is the network endpoint URL that clients should send requests to? (Defined in `<service>` and `<port>` elements.)
3. **HOW** — How should clients communicate with the service? What protocol is used (SOAP over HTTP)? What message format is expected (RPC or Document style)? (Defined in the `<binding>` element.)

Q3. Distinguish between SEI and SIB in JAX-WS development, including the Java annotations used in each. *(6 marks)*

Answer:

In JAX-WS, best practice separates a web service implementation into two distinct components: the **SEI** (Service Endpoint Interface) and the **SIB** (Service Implementation Bean). This separation reflects the principle of separating contract from implementation.

SEI — Service Endpoint Interface:

The SEI is a **Java interface** that declares the operations (methods) that the web service exposes to clients. It acts as the service contract, defining *what* the service can do without specifying *how* it does it. The SEI is the Java equivalent of the WSDL `<portType>` element.

Annotations used on the SEI:

- `@WebService` — marks the interface as a web service definition
- `@SOAPBinding(style = SOAPBinding.Style.RPC)` or `SOAPBinding.Style.DOCUMENT` — specifies the SOAP binding style
- `@WebMethod` — applied to each method, explicitly marking it as an exposed web service operation

```
@WebService
@SOAPBinding(style = SOAPBinding.Style.RPC)
public interface CalculatorInterface {
    @WebMethod
    int add(int a, int b);
}
```

SIB — Service Implementation Bean:

The SIB is a **Java class** that implements the SEI and contains the actual business logic for each declared operation. It is the concrete realisation of the service contract. The SIB can be either a **POJO** (Plain Old Java Object) for lightweight development or a **Stateless Session EJB** (Enterprise Java Bean) for production deployments on application servers, where managed services such as transaction handling, connection pooling, and security are required.

Annotations used on the SIB:

- `@WebService(endpointInterface = "package.InterfaceName")` — links the implementation class to its SEI, forming the complete web service definition

```
@WebService(endpointInterface = "JWS.CalculatorInterface")
public class CalculatorImplementation implements CalculatorInterface {
    @Override
    public int add(int a, int b) {
        return a + b;
    }
}
```

Summary:

Aspect	SEI	SIB
Java type	Interface	Class

Aspect	SEI	SIB
Role	Declares operations (contract)	Implements operations (logic)
Annotation	`@WebService`, `@WebMethod`, `@SOAPBinding`	`@WebService(endpointInterface=...)`
Equivalent WSDL element	`<portType>`	Service implementation
Can be	Only an interface	POJO or Stateless Session EJB

Q4. Write the Java code for publishing a web service named `CalculatorImplementation` at `http://localhost:8081/ws/calculator`. Identify which class and method are used and explain each argument. *(4 marks)*

Answer:

```
package JWS;

import javax.xml.ws.Endpoint;

public class CalculatorWS {
    public static void main(String[] args) {
        Endpoint.publish(
            "http://localhost:8081/ws/calculator",  // Argument 1: publication URL
            new CalculatorImplementation()          // Argument 2: SIB instance
        );
        System.out.println("Service is running at http://localhost:8081/ws/calculator?wsdl");
    }
}
```

Explanation:

- `javax.xml.ws.Endpoint`: This is the JAX-WS class that provides the built-in mechanism for publishing web services. It starts a lightweight embedded HTTP server provided by Java SE — no external application server is needed for development or light-production use.
- `Endpoint.publish(String address, Object implementor)`: The static method that publishes the web service at the specified address.
- **First argument —** `"http://localhost:8081/ws/calculator"`: The publication URL string. This is the address at which the service will listen for incoming SOAP requests. After publishing, clients can access the service at this URL, and the auto-generated WSDL document is accessible at `http://localhost:8081/ws/calculator?wsdl`.
- **Second argument —** `new CalculatorImplementation()`: An instance of the SIB (Service Implementation Bean). This is the object that will handle all incoming SOAP requests, execute the business logic, and generate responses.

Q5. A developer runs a JAX-WS service on JDK 17 and receives a `WebServiceProvider` annotation error. Explain the cause and the solution. *(4 marks)*

Answer:

Cause:

Starting from **Java 9**, the Java platform introduced **Project Jigsaw** — a modular system that divided the monolithic Java SE runtime into named modules. As part of this modularisation, several packages that were previously bundled directly in the JDK were separated out and are no longer available on the default classpath in JDK 9 and above. One of the affected packages is `javax.annotation`, which contains the `@WebServiceProvider` annotation and other annotations used by JAX-WS.

In JDK 8 and earlier, these annotation APIs were bundled directly in the JDK installation files, so no additional configuration was needed. In JDK 9, 11, 17, and above, the `javax.annotation-api` module is not included by default. When code references `@WebServiceProvider` or related annotations, the JVM cannot find the class at runtime, resulting in an annotation-related error.

Solution:

The developer must manually obtain the `javax.annotation-api` JAR file and add it to the project's classpath:

1. **Download the JAR**: Navigate to the Maven Central Repository (https://mvnrepository.com) and search for `javax.annotation-api`. Download the appropriate version (e.g., version 1.3.2).
2. **Add to classpath**: In the IDE (e.g., NetBeans or IntelliJ IDEA), go to Project Properties → Libraries → Classpath and add the downloaded JAR file.

The classpath should then contain both the JAX-WS library and the javax annotation API:

```
Classpath:
  ├── JAX-WS 2.3.5
  └── javax.annotation-api-1.3.2.jar
```

Alternatively, if using a build tool like Maven, add the following dependency to `pom.xml`:

```
<dependency>
    <groupId>javax.annotation</groupId>
    <artifactId>javax.annotation-api</artifactId>
    <version>1.3.2</version>
</dependency>
```

Q6. Describe the five-step pattern used in a JAX-WS Java client to invoke a remote SOAP web service without using `wsimport`. Include the key classes and methods involved. *(10 marks)*

Answer:

When a client developer has access to the web service source code (e.g., when building the provider and consumer in the same project), they can construct a Java client using the JAX-WS API directly without needing `wsimport`. The pattern has five steps:

Step 1 — Create a URL object pointing to the WSDL

```
URL url = new URL("http://localhost:8081/ws/calculator?wsdl");
```

The `java.net.URL` object is used to represent the address of the WSDL document. The `?wsdl` suffix is appended to the service's base URL to retrieve the auto-generated WSDL contract. This URL tells the `Service` factory where to find the service description.

Step 2 — Create a QName identifying the service

```
QName qname = new QName("http://JWS/", "CalculatorImplementationService");
```

`javax.xml.namespace.QName` (Qualified Name) combines a **namespace URI** and a **local name** to uniquely identify the service within the WSDL document. Both values must exactly match what is declared in the WSDL's `<service>` element. The namespace URI corresponds to the target namespace of the WSDL, and the local name is the `name` attribute of the `<service>` element.

Step 3 — Create the Service factory

```
Service service = Service.create(url, qname);
```

`javax.xml.ws.Service` is the JAX-WS factory class. `Service.create(url, qname)` reads the WSDL at the given URL, validates it against the QName, and creates a service factory object that can produce proxy (port) objects.

Step 4 — Extract the Port (proxy object)

```
CalculatorInterface calc = service.getPort(CalculatorInterface.class);
```

`service.getPort(SEI.class)` returns a **proxy object** — a local Java object that implements the SEI interface. When methods are called on this proxy, the JAX-WS runtime automatically serialises the call into a SOAP request message, sends it to the service, receives the SOAP response, deserialises it, and returns the result. From the developer's perspective, it appears as a local method call.

Step 5 — Call the service method

```
int result = calc.add(10, 25);
System.out.println("Result: " + result);
```

The method is called on the proxy object exactly as if it were a local Java method. The underlying SOAP message exchange is completely transparent to the developer. The result is returned as a native Java type.

Complete example:

```
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;

public class CalculatorClient {
    public static void main(String[] args) throws Exception {
        // Step 1: WSDL URL
        URL url = new URL("http://localhost:8081/ws/calculator?wsdl");
        // Step 2: QName (namespace + service name from WSDL)
        QName qname = new QName("http://JWS/", "CalculatorImplementationService");
        // Step 3: Create Service factory
        Service service = Service.create(url, qname);
        // Step 4: Get the port (proxy)
        CalculatorInterface calc = service.getPort(CalculatorInterface.class);
        // Step 5: Call the method
        System.out.println("10 + 25 = " + calc.add(10, 25));
    }
}
```

Output: `10 + 25 = 35`

Q7. Compare the two client approaches demonstrated in the Order Processing Service: the direct QName-based client (OrderClient1) and the wsimport-generated stub client (OrderClient2). What are the advantages and limitations of each? *(6 marks)*

Answer:

OrderClient1 — Direct QName-Based Client:

```
URL wsdlUrl = new URL("http://localhost:8888/ws/orders?wsdl");
QName qname = new QName("http://JWS/", "OrderServiceImplService");
Service service = Service.create(wsdlUrl, qname);
OrderService orderService = service.getPort(OrderService.class);
String response = orderService.placeOrder("Laptop", 2);
```

Advantages:

- No pre-generation step is needed — the client can be written directly after the service is deployed.
- Works well when the service SEI Java code is already available in the same project or shared library.
- Useful for quick testing and development-time consumption.

Limitations:

- Requires the developer to manually read the WSDL to find the correct namespace URI and service name to construct the `QName`.
- The developer must have access to the service's SEI (`OrderService.java`) — not possible if working with a third-party service.
- Verbose: requires explicit URL, QName, Service, and port setup boilerplate.
- If the WSDL changes, the client must be manually updated — no automated regeneration.

OrderClient2 — wsimport-Generated Stub Client:

```
// wsimport generates: OrderServiceImplService.java and OrderService.java
OrderServiceImplService service = new OrderServiceImplService();
OrderService port = service.getOrderServiceImplPort();
System.out.println(port.placeOrder("Tablet", 5));
```

Advantages:

- The client is clean, concise, and readable — only three lines of meaningful code.
- No need to manually look up the WSDL structure: `wsimport` auto-generates type-safe service and port classes.
- The developer does **not** need the original service source code — only the WSDL URL is required. This is essential in cross-team or cross-organisation scenarios.
- Compile-time type safety: the generated `OrderServiceImplService` class ensures the method signatures match what the service actually provides, catching errors at compile time rather than runtime.

Limitations:

- Requires running `wsimport` before the client can be compiled — adds a build-time step.
- If the service WSDL changes (new operations added, parameters renamed), the generated stubs become stale and `wsimport` must be re-run to regenerate them.
- The generated classes can be verbose and numerous for complex services with many operations and data types.

Summary:

Aspect	OrderClient1 (Direct)	OrderClient2 (wsimport)
Setup required	None	Run wsimport first
Source code access needed?	Yes (needs SEI)	No (only WSDL URL)
Code verbosity	High	Low (3 lines)
Type safety	Runtime	Compile-time
WSDL change handling	Manual update	Re-run wsimport
Best for	Same-project development	Cross-team/org integration

Q8. Explain the difference between wsgen and wsimport, specifying the input, output, approach, and typical user of each tool. *(8 marks)*

Answer:

`wsgen` and `wsimport` are two complementary JDK command-line tools that support the Bottom-Up and Top-Down development approaches respectively. They operate at opposite ends of the web service development lifecycle.

---

wsgen (Bottom-Up / Code-First):

`wsgen` is used by the **service provider** who has already written a Java implementation class and wants to turn it into a deployable web service.

- **Input**: A compiled Java `.class` file annotated with `@WebService`.
- **Output**: JAXB (Java Architecture for XML Binding) classes needed for XML marshalling/unmarshalling at runtime. With the `-wsdl` flag, it can also produce a physical WSDL file.

- **Approach**: Bottom-Up — the code comes first, and the service contract (WSDL) is derived from it.
- **Typical user**: The service developer/provider side.
- **When to use**: When you need a physical WSDL file to share with client developers before or after deployment (e.g., to package in an archive, to send to a partner organisation). Note: For simple testing and development, the JAX-WS runtime dynamically generates the equivalent artifacts at `Endpoint.publish()` time, making explicit `wsgen` calls unnecessary for basic scenarios.

```
# Example: generate WSDL from CalculatorImplementation
wsgen -wsdl -cp ./build JWS.CalculatorImplementation
```

---

wsimport (Top-Down / Contract-First):

`wsimport` is used by the **service consumer (client developer)** who has been given a WSDL URL or file and needs to build a Java client without access to the service's source code.

- **Input**: A `.wsdl` file (either a local path or a live URL).
- **Output**: The Service Endpoint Interface (SEI), the generated `Service` class, and JAXB data binding beans (one for each complex type defined in the WSDL's `<types>` section).
- **Approach**: Top-Down — the contract (WSDL) comes first, and the client code is generated from it.
- **Typical user**: The client developer/consumer side.
- **When to use**: Whenever you need to build a Java client for a service for which you only have the WSDL — the standard scenario in cross-team or cross-organisation integration.

```
# Example: generate client stubs for the Calculator service
wsimport -keep -s ./src -p JWS.client http://localhost:8081/ws/calculator?wsdl
```

Comparison:

Feature	wsgen	wsimport
Approach	Bottom-Up (Code-First)	Top-Down (Contract-First)
Primary Input	Compiled `.class` with `@WebService`	`.wsdl` file or URL
Typical User	Service Provider	Service Consumer (Client)
Key Output	JAXB classes, optional WSDL	SEI, Service class, JAXB beans
Required for basic use?	No (runtime generates dynamically)	Yes (for external service clients)

Q9. A client developer receives the WSDL URL `http://api.company.com/ws/inventory?wsdl`. They want to generate Java stubs in the package `com.company.client` and keep the source files. Write the complete wsimport command. *(4 marks)*

Answer:

```
wsimport -keep -s ./src -p com.company.client http://api.company.com/ws/inventory?wsdl
```

Explanation of each flag and argument:

- `wsimport`: The JDK command-line tool for generating client stubs from a WSDL file. Located in `$JAVA_HOME/bin/`.
- `-keep`: Instructs wsimport to retain the generated **Java source files** (`.java`). By default, wsimport only keeps the compiled `.class` files and deletes the intermediate `.java` files. Using `-keep` is important for developers who want to inspect the generated code, add it to version control, or modify it.
- `-s ./src`: Specifies the **output directory** where the generated source files will be placed. Here, `./src` places them in the `src` subdirectory of the current working directory (which is typically the source root of a Java project). Without this flag, files are generated in the current directory.
- `-p com.company.client`: Defines the **Java package name** for all generated classes. This overrides the default package name derived from the WSDL's target namespace. Using `-p` ensures the generated code is placed in a logical, organised package within the project.
- `http://api.company.com/ws/inventory?wsdl`: The URL of the WSDL document. `wsimport` downloads and parses this document, then generates the appropriate Java client stubs from it. This can also be a local file path (e.g., `./inventory.wsdl`).

After running this command, the `./src/com/company/client/` directory will contain the generated SEI, Service class, and any JAXB data binding classes.

Q10. Compare RPC style and Document style SOAP binding in JAX-WS across four dimensions: message structure, supported data types, industry status, and Java annotation. *(8 marks)*

Answer:

1. Message Structure:

In **RPC style**, the SOAP Body element contains a wrapper element named after the operation, and within it are child elements for each parameter — directly mirroring a remote method call. Each parameter is mapped individually as an element in the SOAP Body.

```
<!-- RPC Style SOAP Body for add(10, 25) -->
<soap:Body>
    <ns:add>
        <a>10</a>
        <b>25</b>
    </ns:add>
</soap:Body>
```

In **Document style**, the SOAP Body contains a complete XML document as a single, self-contained message. Rather than mapping parameters to elements, the entire request is packaged as an XML document, which is often more flexible and extensible.

```
<!-- Document Style SOAP Body for placeOrder -->
<soap:Body>
    <ns:placeOrder>
        <item>Laptop</item>
        <quantity>2</quantity>
    </ns:placeOrder>
</soap:Body>
```

2. Supported Data Types:

RPC style is designed for and limited to **simple, primitive data types** such as `String`, `int`, `long`, `double`, and `boolean`. It cannot natively handle complex Java objects or collections without significant additional configuration.

Document style supports both simple types and **rich, complex data types** including custom Java objects, collections, nested objects, and any type that can be mapped to XML Schema via JAXB. This makes it suitable for real-world enterprise services that exchange business objects such as `Order`, `Customer`, or `Invoice`.

3. Industry Status:

RPC style is considered a simpler, older approach. It is easier to understand conceptually (it mimics a direct function call) and is therefore useful for teaching and prototyping. However, it is **not recommended** for production systems due to its limited type support and reduced interoperability with the WS-I Basic Profile.

Document style is the **industry standard**. It is the default SOAP binding style in JAX-WS and is recommended by the WS-I (Web Services Interoperability) organisation. All production web services and the examples from the Order Processing Service in the lecture use Document style.

4. Java Annotation:

Style	Annotation
RPC	`@SOAPBinding(style = SOAPBinding.Style.RPC)`
Document	`@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)` or **omit entirely** (Document is the default)

When `@SOAPBinding` is omitted from the SEI, JAX-WS defaults to Document style automatically.

Q11. Write the complete SEI for a TemperatureConverter web service using Document style that exposes one operation: double celsiusToFahrenheit(double celsius). *(5 marks)*

Answer:

```
package JWS;

import javax.jws.WebMethod;
import javax.jws.WebService;
import javax.jws.soap.SOAPBinding;

/**
 * Service Endpoint Interface (SEI) for the TemperatureConverter web service.
 * Uses Document style (the industry standard, also the JAX-WS default).
 */
@WebService
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public interface TemperatureConverterInterface {

    /**
     * Converts a temperature value from Celsius to Fahrenheit.
     * @param celsius The temperature in degrees Celsius.
     * @return The temperature in degrees Fahrenheit.
     */
    @WebMethod
    double celsiusToFahrenheit(double celsius);
}
```

Annotations explained:

- `@WebService`: Marks this interface as a JAX-WS web service definition, making it the SEI. The JAX-WS runtime uses this annotation to recognise the interface as a service contract.
- `@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)`: Specifies that this service uses Document-style SOAP binding — the industry standard. With Document style, the SOAP Body will contain an XML document representation of the request and response.
- `@WebMethod`: Marks `celsiusToFahrenheit` as an exposed web service operation. Only methods annotated with `@WebMethod` are made available to clients as callable service operations.

The corresponding SIB would implement this interface:

```
@WebService(endpointInterface = "JWS.TemperatureConverterInterface")
public class TemperatureConverterImpl implements TemperatureConverterInterface {
    @Override
    public double celsiusToFahrenheit(double celsius) {
        return (celsius * 9.0 / 5.0) + 32.0;
    }
}
```

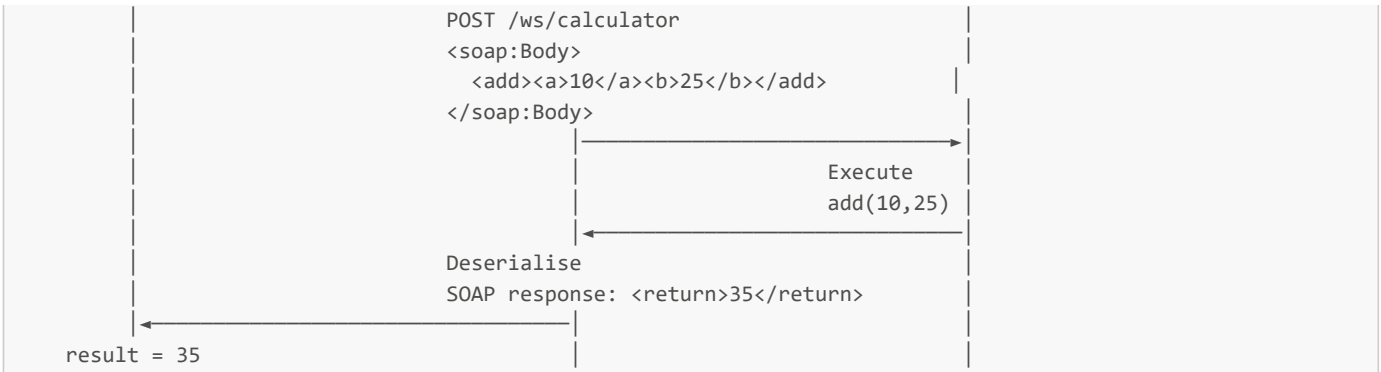
Q12. For the Calculator web service, the WSDL is at `http://localhost:8081/ws/calculator?wsdl`. From the WSDL, the service name = `CalculatorImplementationService`, namespace = `http://JWS/`. Write the Java client code to invoke `add(10, 25)` using the direct (non-wsimport) approach. *(6 marks)*

Answer:

```
package JWS;

import java.net.MalformedURLException;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;

public class CalculatorClient {

    public static void main(String[] args) throws MalformedURLException {

        // Step 1: Create a URL object pointing to the service's WSDL
        URL url = new URL("http://localhost:8081/ws/calculator?wsdl");

        // Step 2: Create a QName using the namespace and service name from the WSDL
        // - 1st arg: target namespace (matches targetNamespace in <definitions>)
        // - 2nd arg: service name (matches name attribute of <service> in WSDL)
        QName qname = new QName("http://JWS/", "CalculatorImplementationService");

        // Step 3: Create the Service factory (reads and parses the WSDL)
        Service service = Service.create(url, qname);

        // Step 4: Get the port - a proxy object implementing CalculatorInterface
        // JAX-WS creates a dynamic proxy that intercepts method calls
        // and converts them to SOAP messages
        CalculatorInterface calc = service.getPort(CalculatorInterface.class);

        // Step 5: Call the web service method exactly like a local Java method
        // Internally: JAX-WS serialises this as a SOAP request, sends it over HTTP,
        // receives the SOAP response, and deserialises the return value
        int result = calc.add(10, 25);

        System.out.println("Result of 10 + 25 = " + result);
        // Output: Result of 10 + 25 = 35
    }
}
```

What happens under the hood:

```
CalculatorClient                    JAX-WS Runtime                CalculatorWS (Server)
       |                                  |                              |
  calc.add(10, 25)                        |                              |
       |----------------------------------|                              |
       |                          Serialise to SOAP:                     |
```



Part II: Essay Questions — Full Model Answers

Essay 1. Discuss the complete architecture of a JAX-WS SOAP-based web service implementation, covering: (a) SEI and SIB roles; (b) required Java annotations; (c) publishing mechanism; (d) the auto-generated WSDL; and (e) Java client consumption with and without wsimport. *(25 marks)*

Introduction

A JAX-WS SOAP-based web service is more than an annotated Java class. It is a structured, multi-component system built around clear separation of contract from implementation, declarative configuration through annotations, and standardised communication through auto-generated WSDL. Understanding the complete architecture — from interface design to client consumption — is essential for building interoperable, maintainable web services.

(a) The Roles of SEI and SIB

The architectural best practice in JAX-WS is to separate the web service into two distinct Java components:

The **Service Endpoint Interface (SEI)** is a Java interface that defines the service's contract. It declares the methods that will be exposed as web service operations, along with any metadata about the SOAP binding style. The SEI is analogous to the `<portType>` element in the WSDL — it defines *what* the service does without specifying *how* it does it. This separation means the contract can be shared with client developers independently of the implementation.

The **Service Implementation Bean (SIB)** is a Java class that provides the concrete realisation of the SEI. It implements every method declared in the interface and contains the actual business logic — database queries, computations, calls to other services, etc. The SIB is linked to its SEI via the `endpointInterface` attribute in its `@WebService` annotation. The SIB can be implemented as a **POJO** for lightweight development scenarios, or as a **Stateless Session EJB** for production deployments on full Java Application Servers (GlassFish, JBoss, WebSphere), where managed services such as transaction management, security enforcement, and connection pooling are required.

This two-component design reflects the principle of **interface-implementation separation** and maps cleanly onto the SOA model: the SEI represents the service's published contract (what the provider offers), and the SIB is the provider's private implementation (how it fulfils that contract).

(b) Java Annotations and Their Significance

JAX-WS uses **declarative annotations** to configure web services without requiring XML deployment descriptors. The key annotations are:

On the SEI:

`@WebService` — applied to the interface, this annotation marks it as a JAX-WS web service definition. It signals to the JAX-WS runtime that this interface represents a service contract.

`@SOAPBinding(style = SOAPBinding.Style.RPC)` or `SOAPBinding.Style.DOCUMENT` — specifies the SOAP message binding style. RPC style maps operations to remote method calls with simple type parameters. Document style (the industry standard and JAX-WS default) sends complete XML documents, supporting complex types. Omitting this annotation defaults to Document style.

`@WebMethod` — applied to each method in the SEI, this annotation explicitly marks the method as an exposed web service operation. Only annotated methods are made available to clients.

```
@WebService
@SOAPBinding(style = SOAPBinding.Style.RPC)
public interface TimeServer2Interface {
    @WebMethod String getTimeAsString();
    @WebMethod long getTimeAsElapsed();
}
```

On the SIB:

`@WebService(endpointInterface = "package.InterfaceName")` — applied to the implementation class, this annotation serves two purposes: it marks the class as a web service and it explicitly links the implementation to its SEI via the `endpointInterface` attribute. This linkage ensures the runtime knows which interface to use when generating the WSDL.

```
@WebService(endpointInterface = "JWS.TimeServer2Interface")
public class TimeServer2Implementation implements TimeServer2Interface {
    @Override
    public String getTimeAsString() { return new Date().toString(); }
    @Override
    public long getTimeAsElapsed() { return new Date().getTime(); }
}
```

These annotations transform ordinary Java classes into fully deployable web services without any additional XML configuration, embodying the "convention over configuration" principle of modern Java development.

(c) The Publishing Mechanism

Once the SEI and SIB are written, the service is published using the `javax.xml.ws.Endpoint` class:

```
Endpoint.publish("http://127.0.0.1:9873/ts", new TimeServer2Implementation());
```

`Endpoint.publish()` starts a **built-in lightweight HTTP server** that is included in Java SE. This server listens on the specified URL for incoming SOAP/HTTP requests. The first argument is the publication URL string; the second is an instance of the SIB that will handle all incoming requests.

This approach is suitable for **development and light production** scenarios. For full production deployments, the SIB (as an EJB) is deployed to a full **Java Application Server** — GlassFish, JBoss/WildFly, IBM WebSphere, or Apache Tomcat — which provides enterprise features: lifecycle management, thread pooling, clustering, failover, and security enforcement.

A note on JDK versions: in JDK 11 and above, the `javax.annotation-api` library is no longer bundled with the JDK due to modularisation. Developers must download and add this JAR to the classpath (available on Maven Central as `javax.annotation:javax.annotation-api:1.3.2`) to avoid runtime annotation errors.

(d) The Auto-Generated WSDL

One of the significant productivity benefits of JAX-WS is that the WSDL is **automatically generated** by the runtime when the service is published. Developers never need to hand-write WSDL XML. The generated WSDL can be viewed by appending `?wsdl` to the service URL:

```
http://localhost:9873/ts?wsdl
```

The generated WSDL contains all the elements of a complete service contract:

- `<types>` — the XML Schema definitions of all data types
- `<message>` — the request and response message definitions for each operation
- `<portType>` — the abstract operations listing (derived from the SEI's `@WebMethod` methods)
- `<binding>` — the SOAP binding (RPC or Document style over HTTP, derived from `@SOAPBinding`)

- `<service>` and `<port>` — the service name and endpoint URL

For example, from the TimeServer, the auto-generated WSDL's `<service>` element contains:

```
<service name="TimeServer2ImplementationService">
    <port name="TimeServer2ImplementationPort" binding="tns:...">
        <soap:address location="http://127.0.0.1:9873/ts"/>
    </port>
</service>
```

This WSDL is the bridge between the provider and the consumer — the machine-readable contract that clients use to understand and invoke the service.

(e) Java Client Consumption: With and Without wsimport

Without wsimport (Direct Pattern):

When the client developer has access to the service's SEI Java class, they can construct a client using the JAX-WS API directly. The five-step pattern involves creating a `URL` pointing to the WSDL, constructing a `QName` with the service's namespace and name (read from the WSDL), creating a `Service` factory, extracting a typed port (proxy), and invoking methods:

```
URL url = new URL("http://localhost:9873/ts?wsdl");
QName qname = new QName("http://JWS/", "TimeServer2ImplementationService");
Service service = Service.create(url, qname);
TimeServer2Interface port = service.getPort(TimeServer2Interface.class);
System.out.println(port.getTimeAsString());
```

This approach requires reading the WSDL to find the correct namespace and service name, and requires access to the SEI class. It is best suited to development scenarios where provider and consumer are in the same project.

With wsimport (Generated Stubs Pattern):

For real-world client development — especially cross-team or cross-organisation scenarios — the client developer may only have the WSDL URL and no access to the service source code. `wsimport` is the solution:

```
wsimport -keep -s ./src -p JWS.client http://localhost:9873/ts?wsdl
```

This generates: the `TimeServer2ImplementationService.java` class (the service factory) and `TimeServer2Interface.java` (the SEI). The client then uses these generated classes in a clean three-step pattern:

```
TimeServer2ImplementationService service = new TimeServer2ImplementationService();
TimeServer2Interface port = service.getTimeServer2ImplementationPort();
System.out.println(port.getTimeAsString());
```

This is significantly cleaner, type-safe (errors caught at compile time), and does not require the developer to inspect the WSDL manually. It embodies the Top-Down (Contract-First) philosophy: the WSDL is the shared contract, and all client code is generated from it.

Complete Development Workflow Diagram:

```
┌──────────────────────────────────────────────────────────────────┐
│                    JAX-WS Development Lifecycle                  │
│                                                                  │
│  PROVIDER SIDE                          CONSUMER SIDE            │
│                                                                  │
│  1. Write SEI                           5. Receive WSDL URL      │
```

```
│     (@WebService, @WebMethod)              from provider              │
│        │                                      │                       │
│  2. Write SIB                            6. Run wsimport              │
│     (@WebService(endpointInterface))        generates: Service + SEI  │
│        │                                      │                       │
│  3. Publish                              7. Write client              │
│     Endpoint.publish(url, sib)               new XxxService()          │
│        │                                     service.getXxxPort()      │
│  4. Test: browser → ?wsdl                    port.method(args)         │
│     Auto-generated WSDL ──────────────────►                            │
└────────────────────────────────────────────────────────────────────────┘
```

Essay 2. Analyse the differences between RPC style and Document style SOAP binding in JAX-WS, covering message construction, data type constraints, industry adoption, annotations, and practical implications for a developer. Illustrate with reference to the lecture examples. *(20 marks)*

Introduction

SOAP binding style determines how Java method calls are translated into XML messages. JAX-WS provides two binding styles — RPC and Document — that differ fundamentally in how they structure SOAP messages, what data types they support, and how they are received in industry practice. Understanding these differences is essential for building professional, interoperable web services.

Conceptual Difference: Message Construction

RPC (Remote Procedure Call) style treats a web service operation as a direct analogue to a function call. When a client invokes `add(10, 25)`, JAX-WS constructs a SOAP Body where the operation name is the outermost wrapper element and the parameters appear as its child elements:

```
<soap:Body>
    <ns:add>
        <a>10</a>
        <b>25</b>
    </ns:add>
</soap:Body>
```

The structure is tightly tied to the method signature. The Body element is named after the operation, and each parameter maps directly to a child element. This design makes RPC-style messages easy to understand but inflexible.

Document style, by contrast, treats the SOAP Body as a container for a **complete, self-describing XML document**. The entire request is packaged as a document:

```
<soap:Body>
    <ns:placeOrder>
        <item>Laptop</item>
        <quantity>2</quantity>
    </ns:placeOrder>
</soap:Body>
```

While the structure may appear similar in simple cases, the crucial difference emerges with complex types. In Document style, the Body can contain arbitrarily complex XML structures mapped from Java objects via JAXB — nested objects, arrays, collections — that RPC style cannot handle. The XML document in the Body is defined by an XML Schema in the WSDL's `<types>` section, making it self-documenting and schema-validated.

Data Type Constraints

This is perhaps the most practically significant difference. RPC style is limited to **simple, primitive data types**: `String`, `int`, `long`, `double`, `boolean`, and their equivalents. This limitation is directly observable in the lecture examples:

- TimeServer2: returns `String` and `long` — simple types, RPC is sufficient.
- HelloWorld: accepts `String`, returns `String` — RPC is appropriate.
- Calculator: accepts `int`, `int`, returns `int` — RPC is appropriate.

However, in the Order Processing Service (Document style), the service accepts `String item` and `int quantity`. While these are still simple, the move to Document style future-proofs the service: if the `placeOrder` operation later needs to accept a complex `Order` object (containing nested `OrderLine` items, customer details, payment information), Document style can accommodate this via JAXB-annotated Java objects mapped to XSD complex types. RPC style would require a complete refactor.

In a real enterprise scenario, a patient records system returning a `Patient` object with name, date of birth, diagnosis list, and medication list would be impossible to implement in RPC style — Document style is the only viable option.

Industry Status and Adoption

RPC style has a conceptual appeal for beginners because it directly mirrors the mental model of a function call — you send parameters, you get a return value. This is why it is used in the first three lecture examples as a teaching tool. However, RPC style is explicitly **not recommended** for production use by the WS-I (Web Services Interoperability) organisation, whose Basic Profile guidelines specify Document style for maximum interoperability.

Document style is the **industry standard** and the JAX-WS default. When `@SOAPBinding` is omitted from an SEI entirely, JAX-WS automatically applies Document style. The WSDL generated from a Document-style service is also more compatible with other platforms (e.g., .NET, Python's Zeep library, PHP's SoapClient), improving cross-platform interoperability.

Annotation Differences

The binding style is controlled entirely by the `@SOAPBinding` annotation on the SEI:

```
// RPC Style (explicit)
@WebService
@SOAPBinding(style = SOAPBinding.Style.RPC)
public interface CalculatorInterface { ... }

// Document Style (explicit)
@WebService
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public interface OrderService { ... }

// Document Style (implicit default — annotation omitted)
@WebService
public interface TemperatureConverterInterface { ... }
```

The SIB and publisher (`Endpoint.publish`) are identical in both styles — the binding style decision is made entirely at the SEI level.

Practical Developer Implications

For a developer building a new production web service, Document style is the correct default choice. The additional setup is minimal (simply use `Style.DOCUMENT` or omit the annotation), and the gains in type flexibility and interoperability are substantial.

For learning and quick prototyping of services that only need to exchange primitive values, RPC style is acceptably simpler and reduces cognitive overhead — which is why the lecture introduces it first.

From a client development perspective, both styles are transparent when using `wsimport`. The generated client stubs behave identically regardless of the binding style — method invocations on the port object look the same to the calling code.

Conclusion

RPC style and Document style represent pedagogical simplicity versus production-grade flexibility. RPC style is a useful teaching abstraction for understanding how method calls map to SOAP messages, but its limitation to simple types makes it unsuitable for enterprise services that exchange complex business objects. Document style, as the JAX-WS default and the WS-I standard, is the correct choice for all

professional SOAP web service development, particularly when the service must handle complex data types, integrate with non-Java clients, or evolve over time without breaking consumers.

Essay 3. Discuss the role of wsgen and wsimport in the SOAP web services development lifecycle, and reflect on practical implications for enterprise teams where provider and consumer may be in different organisations. *(20 marks)*

Introduction

The tools `wsgen` and `wsimport` operationalise the two ends of the SOA development lifecycle: the service provider creating and publishing a service, and the service consumer discovering and binding to it. Understanding when and why to use each tool — and the implications for team-based, cross-organisational development — is central to professional web services practice.

---

wsgen: The Provider's Tool

`wsgen` (Web Service Generator) is the tool used by the service provider after the SIB has been written and compiled. Given a `@WebService`-annotated class, `wsgen` generates:

- **JAXB binding classes**: Java classes that handle the marshalling (Java → XML) and unmarshalling (XML → Java) of data at runtime.
- **A WSDL file** (when using the `-wsdl` flag): a physical, file-based copy of the service contract that can be shared, versioned, and distributed before or independently of deployment.

```
wsgen -wsdl -cp ./build JWS.CalculatorImplementation
# Output: CalculatorImplementationService.wsdl + JAXB classes
```

An important nuance: since Java 6, the **JAX-WS runtime dynamically generates** the equivalent of `wsgen`'s output automatically when `Endpoint.publish()` is called. For simple development and testing, explicit `wsgen` invocations are therefore often unnecessary. However, `wsgen` becomes essential when:

- A physical WSDL file must be produced **before** deployment (e.g., to share with a partner organisation during contract negotiation).
- The WSDL must be packaged inside a WAR or JAR archive for deployment on an application server.
- The provider team needs to version and store the WSDL in source control.

---

wsimport: The Consumer's Tool

`wsimport` (Web Service Importer) is the tool used by the service consumer — the client developer — who has been given a WSDL URL or file but has no access to the service's source code. Given the WSDL, `wsimport` generates:

- The **Service Endpoint Interface (SEI)**: a Java interface matching the operations declared in the WSDL's `<portType>`.
- The **Service class**: a generated subclass of `javax.xml.ws.Service`, named after the service in the WSDL, providing typed `getXxxPort()` methods.
- **JAXB data beans**: Java classes corresponding to any complex types defined in the WSDL's `<types>` / `<xsd:schema>` section.

```
wsimport -keep -s ./src -p com.company.client http://api.provider.com/ws/orders?wsdl
# Output: OrderServiceImplService.java, OrderService.java, JAXB beans
```

The generated Service class encapsulates all the connection details (endpoint URL, namespace, port name), freeing the client developer from reading the WSDL manually. The client code becomes a clean, three-line pattern as demonstrated in the lecture's `CalcClient.java` and `OrderClient2.java`.

The Full Lifecycle: Provider → WSDL → Consumer

```
PROVIDER TEAM                              CONSUMER TEAM
─────────────────                          ──────────────────────
Write SEI + SIB                            Receive WSDL URL from provider
      │                                            │
wsgen (optional: produce physical WSDL)    wsimport -keep -s src -p pkg <WSDL URL>
```

```
      |                                          |
Deploy service                          Generated: Service class, SEI, JAXB beans
      |                                          |
WSDL accessible at ?wsdl URL ──────────────────►|
                                         Write client using generated stubs
                                         new XxxService() → getXxxPort() → call()
```

The WSDL at the `?wsdl` URL is the **shared contract** — the only interface between the two teams. The provider is responsible for maintaining this contract; the consumer regenerates their stubs whenever the WSDL changes.

Enterprise and Cross-Organisational Implications

In a single-organisation project where provider and consumer teams are co-located, the distinction between `wsgen` and `wsimport` is manageable — teams can share source code directly, and the direct `Service.create()` client pattern is usable. However, in cross-organisational enterprise integration — which is the reality of most SOAP web service deployments in banking, healthcare, government, and supply chain — the implications become significantly more serious.

The WSDL as a Binding Contract: When a bank exposes a payment gateway web service to a retail partner, the WSDL is a formally agreed-upon, legally significant document. The provider cannot change it unilaterally without breaking the consumer's generated client stubs and violating the integration agreement. This makes **WSDL versioning** a critical governance concern. Changes should be made through a formal versioning strategy (e.g., publishing a new endpoint at `/v2/payment` while maintaining the old one at `/v1/payment`).

Consumer Independence: The consumer team has no access to the provider's source code — they only have the WSDL URL and the service endpoint. `wsimport` is their only path to building a type-safe Java client. This is exactly the scenario demonstrated by `OrderClient2` in the lecture: the generated `OrderServiceImplService` and `OrderService` classes allow the consumer to call the service without knowing anything about its internal implementation.

Stale Stubs Problem: If the provider updates the service (adds a new operation, changes a parameter type), the consumer's generated stubs become stale. The consumer must re-run `wsimport` against the new WSDL version to regenerate their stubs. This creates a dependency between release cycles of the provider and consumer, which must be managed through API versioning policies and advance notification.

Testing Across Boundaries: Tools like **SoapUI** allow cross-team testing without needing to write client code at all — a developer can send test SOAP requests directly from the WSDL. This is important in cross-organisational integration where the consumer team needs to validate the service before generating client stubs.

Conclusion

`wsgen` and `wsimport` together operationalise the SOA **publish-find-bind** cycle as concrete, JDK-level tools. `wsgen` enables the provider to publish a physical contract; `wsimport` enables the consumer to bind to it programmatically. In enterprise settings, the WSDL they exchange is not merely a technical artefact but a governance document — it defines organisational boundaries, version compatibility, and integration responsibilities. Mastering these tools, and understanding when to use each, is foundational to professional SOAP web services development.

Essay 4 (Applied Design). Build a SOAP-based web service for a hospital system exposing patient record operations: getPatientRecord(int patientId) returning a complex Patient object. *(20 marks)*

(a) Why Document style is more appropriate than RPC style. *(5 marks)*

RPC style is inappropriate for this service for several interconnected reasons.

The `getPatientRecord` operation returns a `Patient` object — a complex domain object containing multiple fields: `name` (String), `dateOfBirth` (Date), `diagnoses` (List<String>), and `medications` (List<String>). RPC style is designed for and limited to simple primitive types (String, int, long, double). It cannot natively serialise complex Java objects or collections into the SOAP Body without breaking the binding. Document style, through JAXB, can map the `Patient` object to a fully defined XML Schema complex type in the WSDL's `<types>` section.

Furthermore, Document style is the **WS-I Basic Profile** recommendation and the **JAX-WS default**, making it the standard for interoperable enterprise web services. Healthcare interoperability standards (such as HL7 and systems interfacing with FHIR gateways) rely on XML document exchange — Document style is architecturally aligned with this.

Finally, Document style's message-level separation allows the service to evolve — adding a `treatmentHistory` field to `Patient` in the future can be done by updating the JAXB class and XSD, with minimal impact on existing consumers. RPC style's tight parameter-mapping makes such evolution far more disruptive.

(b) Design the complete SEI for this service. ***(5 marks)***

```
package hospital.ws;

import javax.jws.WebMethod;
import javax.jws.WebService;
import javax.jws.soap.SOAPBinding;

/**
 * SEI for the Hospital Patient Record Service.
 * Document style used for complex return type (Patient object).
 */
@WebService
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public interface PatientServiceInterface {

    /**
     * Retrieves a patient's complete record by their unique ID.
     * @param patientId  The unique integer identifier of the patient.
     * @return           A Patient object containing full record details.
     */
    @WebMethod
    Patient getPatientRecord(int patientId);
}
```

The `Patient` class would be a JAXB-annotated Java bean:

```
package hospital.ws;

import java.util.List;
import java.util.Date;
import javax.xml.bind.annotation.XmlRootElement;

@XmlRootElement
public class Patient {
    private int id;
    private String name;
    private Date dateOfBirth;
    private List<String> diagnoses;
    private List<String> medications;

    // Getters and setters required by JAXB
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public Date getDateOfBirth() { return dateOfBirth; }
    public void setDateOfBirth(Date dateOfBirth) { this.dateOfBirth = dateOfBirth; }
    public List<String> getDiagnoses() { return diagnoses; }
    public void setDiagnoses(List<String> diagnoses) { this.diagnoses = diagnoses; }
    public List<String> getMedications() { return medications; }
    public void setMedications(List<String> medications) { this.medications = medications; }
}
```

(c) The four-file structure of the full service implementation. ***(4 marks)***

File	Component	Role

File	Component	Role
`PatientServiceInterface.java`	SEI	Declares `@WebService`, `@SOAPBinding(DOCUMENT)`, and `@WebMethod getPatientRecord(int)`. This is the published contract.
`PatientServiceImpl.java`	SIB	`@WebService(endpointInterface="hospital.ws.PatientServiceInterface")`. Implements `getPatientRecord()` — queries the patient database, constructs and returns the `Patient` object. Can be a Stateless EJB for production.
`PatientServicePublisher.java`	Publisher	`Endpoint.publish("http://localhost:9090/ws/patients", new PatientServiceImpl())`. Starts the built-in HTTP server. For production, this is replaced by an application server deployment descriptor.
`PatientClient.java`	Consumer	Uses `wsimport`-generated stubs or the direct `Service.create()` + `QName` pattern to call `getPatientRecord(patientId)`.

```
// PatientServiceImpl.java
@WebService(endpointInterface = "hospital.ws.PatientServiceInterface")
public class PatientServiceImpl implements PatientServiceInterface {
    @Override
    public Patient getPatientRecord(int patientId) {
        // In a real system, this would query a database
        Patient p = new Patient();
        p.setId(patientId);
        p.setName("John Smith");
        p.setDiagnoses(Arrays.asList("Hypertension", "Type 2 Diabetes"));
        p.setMedications(Arrays.asList("Metformin 500mg", "Lisinopril 10mg"));
        return p;
    }
}
```

(d) The process from WSDL generation to client invocation using wsimport. ***(6 marks)***

Step 1 — Publish the service:

Run `PatientServicePublisher.java`. The service publishes at `http://localhost:9090/ws/patients` and the WSDL is accessible at `http://localhost:9090/ws/patients?wsdl`.

Step 2 — Share the WSDL URL with the client developer (third-party Java application team).

Step 3 — Client developer runs wsimport:

```
wsimport -keep -s ./src -p hospital.client http://localhost:9090/ws/patients?wsdl
```

Generated files in `hospital.client` package:

- `PatientServiceImplService.java` — the service class (factory)
- `PatientServiceInterface.java` — the SEI (re-generated, matches the provider's interface)
- `Patient.java` — JAXB data bean for the `Patient` complex type
- `GetPatientRecord.java` and `GetPatientRecordResponse.java` — JAXB wrapper classes for the request/response

Step 4 — Client developer writes the client application:

```
package hospital.client;

public class PatientClient {
    public static void main(String[] args) {
        // Step 1: Initialise the generated Service class
        PatientServiceImplService service = new PatientServiceImplService();

        // Step 2: Get the typed port (proxy)
```

```
        PatientServiceInterface port = service.getPatientServiceImplPort();

        // Step 3: Call the web service method
        Patient patient = port.getPatientRecord(12345);

        // Step 4: Use the returned complex object
        System.out.println("Patient: " + patient.getName());
        System.out.println("Diagnoses: " + patient.getDiagnoses());
        System.out.println("Medications: " + patient.getMedications());
    }
}
```

Output:

```
Patient: John Smith
Diagnoses: [Hypertension, Type 2 Diabetes]
Medications: [Metformin 500mg, Lisinopril 10mg]
```

The `wsimport`-generated stubs completely hide the SOAP message exchange. The `Patient` object is automatically deserialised from the SOAP XML response by JAXB, and the client interacts with it as a native Java object. This demonstrates the full cycle from service design through to transparent cross-system consumption.

End of Chapter 3 Full Model Answers